



ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

Lecture 6

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Functions



Square

Task: calculate the square of a number!

Specification:

In: $x \in \mathbb{R}$

Out: $y \in \mathbb{R}$

Pre: -

Post: $y = x * x$

we would like to isolate
this „complex”
calculation in general

Algorithm:

$y := x * x$

Square

Task: calculate the square of a number!

Specification:

In: $x \in \mathbb{R}$

Out: $y \in \mathbb{R}$

Pre: -

Post: $y = \text{square}(x)$

we hide the calculation in a function
- let's take the calculation of the **square** function as a **separate problem (task)**

Algorithm:

$y := \text{square}(x)$

Square

Mathematics:

$$x \mapsto x^2$$

or:

$$f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^2$$

in our specification:

$$\text{square}: \mathbb{R} \rightarrow \mathbb{R}, \text{square}(n) = n^2$$

Task: Define the „working”
of the square function

Specification:

In: $n \in \mathbb{R}$

Out: $\text{square}(n) \in \mathbb{R}$

Pre: -

Post: $\text{square}(n) = n * n$

input: the parameters of the function (domain)

output: the parametric value of the function
(range)

postcondition: the „calculation”, the relation

Algorithm:

$\text{square}(n:\text{Real}):\text{Real}$

$\text{square} := n * n$

we define the return value with
the name of the function



Square

Task: calculate the square of a number!

Specification:

In: $x \in \mathbb{R}$

Out: $y \in \mathbb{R}$

Fn: $\text{square}: \mathbb{R} \rightarrow \mathbb{R}, \text{square}(n) = n * n$

Pre: -

Post: $y = \text{square}(x)$

formal parameter

actual parameter

Algorithm:

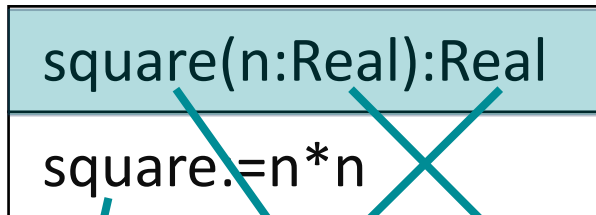
$y := \text{square}(x)$

$\text{square}(n:\text{Real}):\text{Real}$

$\text{square} := n * n$

Square – the code

Task: calculate the square of a number!



```
// square:R->R, square(n)=n*n
static double square(double n) {
    return n * n;
}
```

Square – the code

```
static void Main(string[] args) {  
    Console.WriteLine(3.3 * 3.3);
```

```
    // more general...
```

```
    Console.WriteLine(square(3.3));
```

```
    // Actual parameter: constant
```

```
    double a = square(3.3);
```

```
    // Actual parameter: value of a variable
```

```
    double b = square(a);
```

```
}
```

```
// square:R->R, square(n)=n*n
```

```
static double square(double n) {
```

```
    return n * n;
```

```
}
```

actual parameter

formal parameter

square(n:Real):Real

square:=n*n



Maximum

Task: return the larger of two numbers

Specification:

In: $a \in \mathbb{Z}$, $b \in \mathbb{Z}$

Out: $m \in \mathbb{Z}$

Pre: -

Post: $m = \max(a, b)$

What can be seen from postcondition?

1. the name of the function: max
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 1
5. type of the return value(s): Integer

Maximum

Task: return the larger of two numbers

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $m \in \mathbb{Z}$

Fn: $\text{max} : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}, \text{max}(a, b) = \begin{cases} a, & \text{if } a \geq b; \\ b & \text{else} \end{cases}$

Pre: -

Post: $m = \text{max}(a, b)$

What can be seen from postcondition?

1. the name of the function: max
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 1
5. type of the return value(s): Integer



Maximum

Task: return the larger of two numbers

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $m \in \mathbb{Z}$

Fn: $\text{max} : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z},$
 $\text{max}(a, b) = \{a, \text{ if } a \geq b;$
 $\quad \quad \quad b \text{ else}\}$

Pre: -

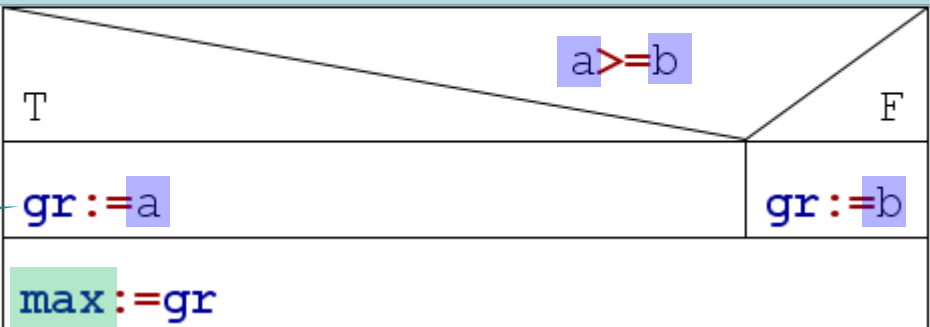
Post: $m = \text{max}(a, b)$

gr: helper variable (Integer)

Algorithm:

$m := \text{max}(a, b)$

$\text{max}(a : \text{Integer}, b : \text{Integer}) : \text{Integer}$



Maximum – the code

```
static void Main(string[] args) {  
    // max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
    Console.WriteLine(max(3, 7));  
}  
// max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
static int max(int a, int b) {  
    int gr;  
    if (a >= b) {  
        gr = a;  
    }  
    else {  
        gr = b;  
    }  
    return gr;}  
}
```

max(a:Integer, b:Integer):Integer

a >= b	
T	F
gr := a	gr := b
max := gr	

Maximum – the code

```
static void Main(string[] args) {  
    // max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
    Console.WriteLine(max(3, 7));  
}  
// max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
static int max(int a, int b) {  
    if (a >= b) {  
        return a;  
    }  
    else {  
        return b;  
    }  
}
```

max(a:Integer, b:Integer):Integer

a >= b	
T	F
max := a	max := b

not really „clear” code – we have 2 „return points”:
more potential for error

Maximum – the code

```
static void Main(string[] args) {  
    // max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
    Console.WriteLine(max(3, 7));  
}  
// max:ZxZ->Z, max(a, b)={a, if a>=b; b else}  
static int max(int a, int b) {  
    return a >= b ? a : b;  
}
```

conditional expression – language specific
(C#: ternary operator ?)

max(a:Integer, b:Integer):Integer

a >= b	
T	F
gr := a	gr := b
max := gr	

Increasing a value

Task: increase a value of a variable by a given d

Specification:

In: $n \in \mathbb{Z}, d \in \mathbb{Z}$

Out: $n' \in \mathbb{Z}$

The same data, but the state is changed
state = value

Pre: -

Post: $n' = \text{incr}(n, d)$

```
// example  
int a = 10;  
// prefer to use:  
incr(a, 8);  
// not so:  
a = incr(a, 8);
```

What can be seen from postcondition?

1. the name of the function: `incr`
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 1
5. type of the return value(s): Integer

Increasing a value

```
// example  
int a = 10;  
// prefer to use:  
incr(a, 8);  
// not so:  
a = incr(a, 8);
```

Task: increase a value of a variable by a given d

Specification:

In: $n \in \mathbb{Z}, d \in \mathbb{Z}$

Out: $n' \in \mathbb{Z}$

Fn: $\text{incr}: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z},$
 $\text{incr}(\text{numb}, \text{byd}) = \text{numb} + \text{byd}$

Pre: -

Post: $n' = \text{incr}(n, d)$

What can be seen from postcondition?

1. the name of the function: `incr`
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 1
5. type of the return value(s): Integer

Increasing a value

```
// example
int a = 10;
// prefer to use:
incr(a, 8);
// not so:
a = incr(a, 8);
```

Task: increase a value of a variable by a given d

Specification:

In: $n \in \mathbb{Z}, d \in \mathbb{Z}$

Out: $n' \in \mathbb{Z}$

Fn: `incr`: $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$,
`incr(num, byd) = num + byd`

Pre: -

Post: $n' = \text{incr}(n, d)$

Algorithm:

```
incr(a, 8)
```

```
incr(var numb: Integer, byd: Integer)
```

```
numb := numb + byd
```

the value of the parameter will change

Increasing a value – the code

we don't have
return value: void

```
incr(var numb:Integer, byd:Integer)
```

```
numb:=numb+byd
```

```
static void Main(string[] args) {  
    int a = 10;  
    Console.WriteLine("before: {0}", a);  
    novel(a, 8);  
    Console.WriteLine("after: {0}", a);  
}
```

```
static void incr(int numb, int byd) {  
    numb = numb + byd;  
}
```

before: 10
after: 10

WRONG

Passing the parameter by value: a local identifier is created from the formal parameter, and the value of the actual parameter is copied into it. Thus, changing the parameter within the function body won't affect the actual parameter.



Increasing a value – the code

```
incr(var numb:Integer, byd:Integer)
```

```
numb:=numb+byd
```

```
static void Main(string[] args) {  
    int a = 10;  
    Console.WriteLine(„before: {0}”, a);  
    novel(ref a, 8);  
    Console.WriteLine(„after: {0}”, a);  
}
```

```
static void incr(ref int numb, int byd) {  
    numb = numb + byd;  
}
```

before: 10
after: 18

WORKS

Passing the parameter by reference: the address (memory reference) of the actual parameter is copied into the formal parameter, so it will be available through its local name



Exchange the values

Task: exchange the value of two variables!

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $a' \in \mathbb{Z}, b' \in \mathbb{Z}$

The same data, but the state is changed
state = value

Pre: -

Post: $a' = b$ and $b' = a$

```
// example
int a = 10, b = 20;
// exchange:
Console.WriteLine(„before: {0}, {1}”, a, b);
csere(a, b);
Console.WriteLine(„after: {0}, {1}”, a, b);
```

Exchange the values

Task: exchange the value of two variables!

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $a' \in \mathbb{Z}, b' \in \mathbb{Z}$

```
// example
int a = 10, b = 20;
// exchange:
Console.WriteLine(„before: {0}, {1}", a, b);
csere(a, b);
Console.WriteLine(„after: {0}, {1}", a, b);
```

Pre: -

Post: $a', b' = \text{swap}(a, b)$

What can be seen from postcondition?

1. the name of the function: swap
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 2
5. type of the return value(s): Integer, Integer

Exchange the values

Task: exchange the value of two variables!

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $a' \in \mathbb{Z}, b' \in \mathbb{Z}$

Fn: $\text{swap}: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z} \times \mathbb{Z}$,
 $\text{swap}(x, y) = (y, x)$

Pre: -

Post: $a', b' = \text{swap}(a, b)$

```
// example
int a = 10, b = 20;
// exchange:
Console.WriteLine(„before: {0}, {1}", a, b);
csere(a, b);
Console.WriteLine(„after: {0}, {1}", a, b);
```

What can be seen from postcondition?

1. the name of the function: swap
2. number of parameters: 2
3. type of the parameters: Integer, Integer
4. number of return value(s): 2
5. type of the return value(s): Integer, Integer

Exchange the values

Task: exchange the value of two variables!

Specification:

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $a' \in \mathbb{Z}, b' \in \mathbb{Z}$

Fn: $\text{swap}: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z} \times \mathbb{Z},$
 $\text{swap}(x, y) = (y, x)$

Pre: -

Post: $a', b' = \text{swap}(a, b)$

Algorithm:

```
swap(a, b)
```

```
swap(var x:Integer, var y:Integer)
```

```
z := x
```

```
x := y
```

```
y := z
```

Exchange the values – the code

```
static void Main(string[] args) {  
    int a = 10, b = 8;  
    Console.WriteLine(„before: {0}, {1}", a, b);  
    swap(ref a, ref b);  
    Console.WriteLine(„after: {0}, {1}", a, b);  
}
```

```
static void swap(ref int x, ref int y) {  
    int z = x;  
    x = y;  
    y = z;  
}
```

before: 10, 8
after: 8, 10

```
swap(var x:Integer, var y:Integer)
```

```
z:=x
```

```
x:=y
```

```
y:=z
```



Functions – concepts

Block: The program text between { and the belonging closing }

Scope: The scope of an X identifier is the part of the program text (not necessarily contiguous) where one can refer to X.

- Scope is defined by the structure of blocks. If two blocks have common parts, then one of those entirely contains the other.
- The scope of an identifier reaches from the following character of the declaration up to the closing } of the block, except for the blocks (and it's descendants) where it was re-declared.

Functions – concepts

Scope: An identifier that is referred to at a certain place is **local**, if it was declared in the smallest block containing the reference. An identifier is **global** (for the given block) if it is not local.

Lifetime: The lifetime of all the variables declared in block B lasts from entering the block to the completion of the last instruction of the block



Functions – concepts

Passing parameters:

- **By value** – a local identifier is created from the formal parameter, and the value of the actual parameter is copied into it. Thus, changing the parameter within the function body won't affect the actual parameter.

```
static int max(int x, int y)
```

input parameters

- **By reference** – the address (memory reference) of the actual parameter is copied into the formal parameter, so it will be available through its local name.

```
static void max(int x, int y, ref int max_xy)
```

```
static void max(int x, int y, out int max_xy)
```

in/output parameters

ref: the parameter can be modified

out: the parameter needs to be modified (it didn't have value before using)

Solve a task together...



Train faster than the previous one

We know the travel times of N trains. Write a program that identifies a train that is faster than the previous one.

faster = less time!

Input

Output

6

3

the index of the train (faster than the previous one) – if we have more, than the first;

118

200

199

116

200

122

the number of trains

each line: an integer, the travel time of the i. train



Train faster than the previous one

Is there any train that is faster... - if yes, which one.

→ search pattern

	6
1	118
2	200
3	199
4	116
5	200
6	122

Specification:

In: $n \in \mathbb{N}$, $ttime \in \mathbb{N}[1..n]$

Out: $exists \in \mathbb{L}$, $ind \in \mathbb{N}$

Pre: -

Post: $(exists, ind) = \text{SEARCH}(i=2..n, ttime[i] < ttime[i-1])$

what are the endpoints of the interval?

what are the names of the output data?

what is the attribute that we are looking for?



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

Train faster than the previous one

Pattern (search)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}, \text{ind} \in \mathbb{Z}$

Pre: -

Post: $(\text{exists}, \text{ind}) =$
 $\text{SEARCH}(i=b..e, A(i))$

Specific task

In: $n \in \mathbb{N}, \text{ttime} \in \mathbb{N}[1..n]$

Out: $\text{exists} \in \mathbb{L}, \text{wind} \in \mathbb{N}$

Pre: -

Post: $(\text{exists}, \text{wind}) =$
 $\text{SEARCH}(i=2..n, \text{ttime}[i] < \text{ttime}[i-1])$

$\text{exists}, \text{ind}$	$\sim$	$\text{exists}, \text{wind}$
$b..e$	$\sim$	$2..n$
$A(i)$	$\sim$	$\text{ttime}[i] < \text{ttime}[i-1]$

$\text{ind} := b$

$\text{ind} \leq e$ and not $A(\text{ind})$

$\text{ind} := \text{ind} + 1$

$\text{exists} := (\text{ind} \leq e)$

$\text{wind} := 2$

$\text{wind} \leq n$ and not $(\text{ttime}[\text{wind}] < \text{ttime}[\text{wind}-1])$

$\text{wind} := \text{wind} + 1$

$\text{exists} := (\text{wind} \leq n)$



Train faster than the previous one the code – monolithic

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    Console.Write("n = ");  
    int.TryParse(Console.ReadLine(), out n);  
    ttime = new int[n];  
    for (int i = 0; i < n; i++) {  
        Console.Write("{0}. traveling time = ", i+1);  
        int.TryParse(Console.ReadLine(), out ttime[i]);  
    }  
    // Solving the problem (algorithm)  
    wind = 1;  
    while (wind < n && !(ttime[wind] < mido[wind - 1])) {  
        wind = wind + 1;  
    }  
    exists = (wind < n);  
    // Writing the output  
    if (exists) {  
        Console.WriteLine(„The {0}. train is faster than the previous one.", wind);  
    }  
    else {  
        Console.WriteLine(„There is no such train.");  
    }  
}
```

Train faster than the previous one the code – with functions



```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(n,ttime);  
  
    // Solving the problem (algorithm)  
    search_train(n,ttime,exists,wind);  
    //OR  
    (exists,wind)=search_train(n,ttime);  
  
    // Writing the output  
    writeOut(exists,wind);  
  
}
```

Train faster than the previous one the code – with functions

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(n,ttime);  
  
    // ...  
}
```

we don't have return value: void

passing by reference

```
static void readIn(out int n, out int[] where) {  
    // reading the input  
    Console.Error.Write("n = ");  
    int.TryParse(Console.ReadLine(), out n);  
    where = new int[n];  
    for (int i = 0; i < n; i++) {  
        Console.Error.Write("{0}. traveling time = ", i+1);  
        int.TryParse(Console.ReadLine(), out where[i]);  
    }  
}
```

Train faster than the previous one the code – with functions

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(out n, out ttime);  
    search_train1(n, ttime, out exists, out wind);  
  
    // ...  
}
```

we don't have return value: void

passing by reference

passing by value

arrays are always passing by reference!

```
static void search_train1(int n, int[] ttime, out bool exists, out int wind)  
{  
    wind = 1;  
    while (wind < n && !(ttime[wind] < ttime[wind - 1])) {  
        wind = wind + 1;  
    }  
    exists = (wind < n);  
}
```

Train faster than the previous one the code – with functions

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(out n, out ttime);  
    (exists, wind) = search_train2(n, ttime);  
  
    // ...  
}
```

tuple

```
static (bool exists, int wind) search_train2(int n, int[] ttime) {  
    int wind;  
    bool exists;  
    wind = 1;  
    while (wind < n && !(ttime[wind] < ttime[wind - 1])) {  
        wind = wind + 1;  
    }  
    exists = (wind < n);  
  
    return (exists, wind);  
}
```

Specification:

In: $n \in \mathbb{N}$, $ttime \in \mathbb{N}[1..n]$

Out: $exists \in \mathbb{L}$, $wind \in \mathbb{N}$

Pre: -

Post: $(exists, wind) =$

SEARCH($i=2..n$, $ttime[i] < ttime[i-1]$)

the solution can be considered as a function:

$search_train: \mathbb{N} \times \mathbb{N}[] \rightarrow \mathbb{L} \times \mathbb{N}$

$(exists, wind) = search_train(n, ttime)$

Train faster than the previous one the code – with functions

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(out n, out ttime);  
    (exists, wind) = search_train2(n, ttime);  
    writeOut(exists, wind);  
  
    // ...  
}  
static void writeOut(bool exists, int wind) {  
    if (exists) {  
        Console.WriteLine(wind+1);  
    }  
    else {  
        Console.WriteLine(-1);  
    }  
}
```

user friendly

Train faster than the previous one

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] ttime;  
    bool exists;  
    int wind;  
    // Reading the input  
    readIn(out ttime);  
    (exists,wind)=search_train2(ttime);  
    writeOut(exists,wind);  
  
    // ...  
}  
static void readIn(out int[] where) {  
    int n;  
    Console.Error.Write("n = ");  
    int.TryParse(Console.ReadLine(), out n);  
    where = new int[n];  
    for (int i = 0; i < n; i++) {  
        Console.Error.Write("{0}. traveling time = ", i+1);  
        int.TryParse(Console.ReadLine(), out where[i]);  
    }  
}  
static (bool exists, int wind) search_train2(int[] ttime) {  
    int wind;  
    bool exists;  
    wind = 1;  
    while (wind < ttime.Length && !(ttime[wind] < ttime[wind - 1])) {  
        wind = wind + 1;  
    }  
    exists = (wind < ttime.Length );  
  
    return (exists,wind);  
}
```

the array (in C#) contains its length – we can „get“ it:
n=ttime.length;

Dynamic array



Array – the „problem”

- need to allocate memory at the beginning
- idea: only if we know the n (the size)

```
int[] ttime;  
...  
Console.Error.Write("n = ");  
int.TryParse(Console.ReadLine(), out n);  
ttime = new int[n];
```

- if we have an array in the output – what we know about the size?
 - we need to allocate the maximum – and then don't use all places
 - Multiple item selection, combined patterns,...

Dynamic array

- A dynamic array is used to represent a sequence of variable number of elements
 - It can be resized during the execution as required.
 - This is exactly what we need for the multiple item selection (MIS)

- Specification

Out: ~~$\text{ent} \in \mathbb{N}$~~ , $y \in \mathbb{N}[1..\text{ent}]$

Post: $\forall i \in [1..\text{length}(y)] : (A(y[i]))$

- Algorithm

Variable $y : \text{Array}[1..\text{ent} : \text{Integer}]$

- Code

```
List<int> y = new List<int>();  
y.Add(1);  
y.Add(2);  
Console.WriteLine(y[0]);    // 1  
Console.WriteLine(y.Count); // 2
```

1. get a „start address”
2. the length is 0
3. new operations:
 - length
 - at the end (Add)

Dynamic array

- the *i*th item doesn't exist at the beginning

```
List<int> y = new List<int>();
```

```
y[0]=1;
```

Doesn't exist the 1st item!

referencing to the 1st item

```
y.Add(1);
```

```
y[0]=5;
```

referencing to the 1st item
safeguard against a non-
existent element

```
y.ElementAt(0);
```

```
Console.WriteLine(y.Count);
```

the length of the List

Dynamic array – the code

```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    List<int> ttime = new List<int>;  
  
    // ...  
  
    int n;  
    Console.Error.Write("n = ");  
    int.TryParse(Console.ReadLine(), out n);  
    int hv;  
    for (int i = 0; i < n; i++) {  
        Console.Error.Write("{0}. traveling time = ", i+1);  
        int.TryParse(Console.ReadLine(), out hv);  
        ttime.Add(hv);  
    }  
}
```





ELTE
FACULTY OF
INFORMATICS

```
(function repeat() {  
    eat();  
    sleep();  
    coffeeLoader();  
    code();  
    repeat();  
})();
```

Thank you for your attention!